

Take-Home Assignment: Algorithmic Trading Strategy Development & Backtesting

Role: Quantitative / Trading Developer **Issued by:** Hiring Team **Time allowed:** 7 days from the date you receive this document **Submission:** Private GitHub repository (details in Section 7)

1. Purpose of This Assignment

The primary goal of this assignment is **not** to test whether you are already an expert in algorithmic trading. It is to evaluate **how quickly you can learn a new domain and apply it to a real, working project.**

We want to see how you:

- Pick up unfamiliar tools and libraries and become productive fast
- Translate a trading idea into clean, working, well-structured code
- Validate your own work honestly using proper testing methodology
- Communicate and explain technical decisions clearly

If you have never built a trading strategy before, that is completely fine — that is exactly the situation this assignment is designed to assess.

2. The Task

Build, backtest, and validate an **algorithmic trading strategy** for a publicly traded stock.

You will:

1. **Choose any stock symbol** of your choice (e.g., AAPL, MSFT, TSLA, INFY, RELIANCE, etc.).
2. **Download historical price data** for that symbol from **Yahoo Finance** (the `yfinance` Python library is the easiest route) or any other reliable data source.
3. **Design a trading strategy** — entry rules, exit rules, and any risk management logic. The strategy logic is your own; be creative, but keep it explainable.
4. **Implement and backtest the strategy in Backtrader**, a popular open-source Python backtesting framework.
5. **Run a Walk-Forward Analysis** to confirm the strategy is not simply overfitted to one period of data.
6. **Compute a Robustness Score** for your strategy. **Your strategy's robustness score must be greater than 75.**

Note: A strategy that shows a modest but *robust and honest* return is far more valuable to us than one showing a spectacular but overfitted return. Do not curve-fit.

3. Technical Requirements

Requirement	Detail
Language	Python 3.x
Backtesting framework	Backtrader (mandatory)
Data source	Yahoo Finance (yfinance) or any other reliable historical data source
Instrument	Any one publicly traded stock symbol
History length	Use enough historical data to make walk-forward analysis meaningful — at least 5 years of daily data is recommended
Starting capital	You may choose your own starting capital. State it clearly and report all returns as a percentage of that capital so results are comparable
Code quality	Clean, readable, commented, and runnable end-to-end with clear setup instructions

4. Key Concepts Explained

If these terms are new to you, here is what we expect — researching and understanding them is part of the assignment.

4.1 Backtesting

Running your strategy against historical data to simulate how it *would* have performed. Backtrader handles the trade simulation, position sizing, commissions, and performance metrics for you.

4.2 Walk-Forward Analysis (WFA)

A more honest form of testing than a single backtest. Instead of optimizing and testing on the same data, you:

1. Split your history into multiple sequential segments.
2. **Optimize** strategy parameters on an *in-sample* window (e.g., 2 years).
3. **Test** those parameters on the *next, unseen out-of-sample* window (e.g., 6 months).
4. Roll the window forward and repeat across the full history.

This reveals whether the strategy holds up on data it was **not** tuned on — i.e., whether it would have worked in real life.

4.3 Robustness Score (must be > 75)

A single 0–100 score summarizing how dependable your strategy is. You may use an existing methodology or define your own, but it **must** account for factors such as:

- **Walk-forward efficiency** — out-of-sample performance vs. in-sample performance (less degradation = better)
- **Consistency** — does it perform across most windows, or only one lucky period?
- **Parameter sensitivity** — does a small change in parameters destroy the results? (stable = robust)
- **Drawdown control** — are losses contained?

You must clearly document how you calculated your robustness score in the README. A score above 75 with a transparent, defensible methodology is the requirement.

5. Required Deliverables in Your Submission

Your submission **must explicitly report** the following four items (in the repository README):

1. **Percentage Return on Capital** — total return as a % of your stated starting capital
2. **Maximum Drawdown** — the largest peak-to-trough decline, as a %
3. **Walk-Forward Analysis Score** — your WFA result / efficiency
4. **GitHub Repository URL** — link to your private repo (see Section 7)

A suggested results summary table for your README:

```
## Results Summary
```

Metric	Value
-----	-----
Stock Symbol	e.g. AAPL
Backtest Period	YYYY-YYYY
Starting Capital	\$XX,XXX
Percentage Return on Capital	XX.X %
Maximum Drawdown	XX.X %
Walk-Forward Analysis Score	XX
Robustness Score	XX (> 75)

6. What Your Repository Should Contain

- All strategy and backtesting **source code** (Backtrader implementation)
- The **walk-forward analysis** code/script
- The **robustness score** calculation
- A clear **README.md** containing:
 - Setup & run instructions (dependencies, how to reproduce your results)
 - A description of your strategy logic (entry/exit/risk rules)
 - The results summary table from Section 5
 - Your robustness score methodology

- A short note on what you learned during this assignment
- A `requirements.txt` (or equivalent) so we can run it
- (Optional but appreciated) equity curve / drawdown charts

7. Submission Instructions

1. Create a **private GitHub repository** for your work.
2. Grant **read access** to the GitHub user `markheris34-svg` so our team can review your code.
(Repository → Settings → Collaborators → add `markheris34-svg`)
3. Reply to this email / message with your **repository URL**.
4. Submit **within 7 days** of receiving this assignment.

8. Use of AI Tools

You are free to use any AI tools (ChatGPT, Claude, Copilot, etc.) to help you learn and build faster — we encourage working the way you would work on a real project.

However: you must **fully understand every line of your submission**. Shortlisted candidates will be asked to **explain their complete code, design choices, and results in a live technical interview**. Being unable to explain your own submission will end the process. Use AI to *learn faster*, not to *skip understanding*.

9. How You Will Be Evaluated

Area	What we look for
Learning speed	Evidence you picked up Backtrader, walk-forward analysis, and robustness testing quickly and correctly
Correctness	The code runs, the methodology is sound, results are honest and reproducible
Robustness	Robustness score genuinely above 75 with a transparent methodology — not overfitting
Code quality	Clean, readable, well-structured, documented
Performance	Percentage return on capital (see note below)
Communication	Clear README and the ability to explain everything in the interview

Interview shortlisting: We will **interview the candidates whose strategies achieve the highest percentage return on capital** — provided their robustness score exceeds 75 and the results are genuine and reproducible. A high return with a weak or overfitted methodology will not qualify.

10. Quick-Start Hints

- `pip install backtrader yfinance pandas matplotlib`
 - Start simple — get a basic strategy backtesting in Backtrader **first**, then layer on walk-forward analysis and the robustness score.
 - Keep your strategy logic explainable. A clean moving-average / momentum / mean-reversion idea done *rigorously* beats a complex black box.
 - Be honest with yourself: if walk-forward results collapse, that is a finding worth reporting — fix the strategy, don't hide it.
 - Document as you go so your README is effortless at the end.
-

Good luck — we're looking forward to seeing how you approach this.